print()

Now that we know how to read various characters coming in from the Serial input, let's look at a few functions that will send information out through Serial, beginning with the `print()` function. These functions work a little differently from the `read()` function, so we will try to be thorough at the risk of putting our readers to sleep. The syntax for this function follows:

```
Serial.print(data);
```

The `print()` function is used for printing ASCII characters to the connected serial device. The data we specify as a parameter can take many different forms, so we need to make sure that the data we want to send is in a format of our choosing. The `print()` function makes the general assumption that if you specify a value like 33, what you really want to send are the characters `'3'` and `'3'` not the ASCII character code for `'!'`. There's another function for that, but first, Table 10-3 looks at some of the different examples of various statements and what output will be displayed.

Table 10-3. *Examples of Serial.print()*

Statement	Output
Serial.print(33);	33
Serial.print(3.1459);	3.15
Serial.print('A');	A
Serial.print("a brave new world");	a brave new world
Serial.print(3.1459, 4);	3.1459
Serial.print(char(33));	!

These examples begin with the default behavior of the `print()` function in that by providing a numerical value to the function, the function will display each of the characters that make up each digit of the number. It is important to note that our example does not print the integer `33` but rather the characters `'3'` and `'3'`. The second example shows that by default, the `print()` function will print to two decimal places. Characters like the letter `'a'` are enclosed with single quotes while text strings such as `"a brave new world"` are bracketed in double quotes. Finally, we bend the function a little by specifying additional decimal places for displaying floats and the last example shows how to force a `char` data type on a numerical value to display the ASCII character for that value.

Each time the `print()` function is called, data is displayed in one continuous line. To create spacing between multiple values we might use one of the three following statements:

```
Serial.print("     ");
Serial.print(char(9));
Serial.print ('\t');
```

The first of these three statements prints the text string of five spaces. Instead, we could use the ASCII code for the escape sequence for tab. By forcing the `char()` data type, the `print()` function will tabulate the next printed data. Alternatively, we could use the `'/t'` escape sequence to create a tab as